

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 2- ANALYTICAL PROBLEM SOLVING

Course Code:	CSA1205	Course Title:	Computer Architecture
CO Assessed:	CO2	Assessment:	ASSESSMENT TOOL 2- ANALYTICAL PROBLEM SOLVING
Weightage:	45%	Total Marks:	25
CO2 Statement:	Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)		
Student Name:	G.Saranraj	Reg.No.:	192511182
Department:		Date:	

ANNEXURE A

1. A Booth's multiplication program produces an incorrect product for the inputs -13×9 . Debug the algorithm by identifying the incorrect shift or addition/subtraction step and optimize the implementation to reduce execution cycles.
2. A processor implementing the **restoring division algorithm** generates an incorrect quotient for $29 \div 5$. Debug the iteration sequence, identify the faulty restore operation, and propose an optimized correction.
3. During the execution of the **non-restoring division algorithm**, the quotient bits are incorrect after the fourth iteration. Identify the logic error, correct the algorithm, and verify the final quotient and remainder.
4. A Booth multiplier performs arithmetic right shifts as logical shifts, resulting in an incorrect signed multiplication result. Debug the issue and explain how arithmetic shifting improves correctness.
5. A floating-point addition program based on the **IEEE 754 single-precision standard** gives an incorrect result because the exponents are not aligned properly. Identify the bug and optimize the alignment process

Code of Conduct:

I Saranraj.G / 192511182 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:**MARKS ALLOCATION**

	Problem Understanding (20%)	Method Selection (20%)	Solution Process (25%)	Accuracy of Calculation (20%)	Interpretation of Results (15%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

ANNEXURE A – SOLUTIONS

Student Name: Saranraj.G Reg. No.: 192511182

Question 1: Debugging Booth's Multiplication (-13×9)

Problem Understanding:

A Booth's multiplication routine is producing an incorrect product for $M = -13$ and $Q = +9$. Using 5-bit registers (needed since -13 requires 5 bits): $-13 = 10011$, so $-M = 01101$, and $+9 = 01001$. We must re-run the algorithm correctly, locate the faulty step, and verify the true product.

Step 1 – Correct Cycle-by-Cycle Execution

Initial: $A=00000$, $Q=01001$, $Q-1=0$.

Cycle 1: $Q_0Q_{-1}=1,0 \rightarrow A=A-M=00000+01101=01101$. Arithmetic shift $\rightarrow A=00110$, $Q=10100$, $Q-1=1$.

Cycle 2: $Q_0Q_{-1}=0,1 \rightarrow A=A+M=00110+10011=11001$. Arithmetic shift $\rightarrow A=11100$, $Q=11010$, $Q-1=0$.

Cycle 3: $Q_0Q_{-1}=0,0 \rightarrow$ no arithmetic operation, shift only. Arithmetic shift $\rightarrow A=11110$, $Q=01101$, $Q-1=0$.

Cycle 4: $Q_0Q_{-1}=1,0 \rightarrow A=A-M=11110+01101=01011$. Arithmetic shift $\rightarrow A=00101$, $Q=10110$, $Q-1=1$.

Cycle 5: $Q_0Q_{-1}=0,1 \rightarrow A=A+M=00101+10011=11000$. Arithmetic shift $\rightarrow A=11100$, $Q=01011$, $Q-1=0$.

Final product = $A:Q = 1110001011$ (10 bits) = -117 in decimal, which correctly matches $-13 \times 9 = -117$.

Step 2 – Locating the Bug

The faulty implementation was found to perform a LOGICAL right shift of the $[A \ Q \ Q-1]$ register instead of an ARITHMETIC right shift. This only causes visible corruption in cycles where A is negative ($MSB = 1$) — for example, Cycle 2 above where $A = 11001$ before shifting. A correct arithmetic shift must replicate (copy) the sign bit into the new MSB, giving $A = 11100$ after the shift. The buggy logical shift instead inserts a 0 into the new MSB, giving $A = 01100$ — this silently converts a negative partial product into a positive one, corrupting the sign of every subsequent add/subtract decision and producing a final product far from -117 (a logical-shift trace on the same inputs drifts to an incorrect magnitude with the wrong sign).

Step 3 – Correction and Optimization

Fix: replace the logical right-shift instruction with an arithmetic right-shift that sign-extends from A 's current MSB before each shift of the combined $[A \ Q \ Q-1]$ register, as done in Step 1 above.

Optimization to reduce cycles: plain Booth's algorithm already skips arithmetic operations whenever $Q_0Q_{-1} = 00$ or 11 (as seen in Cycle 3, a shift-only cycle), but it still needs one cycle per bit. A faster implementation uses Radix-4 (Modified) Booth Recoding, which

inspects 3 overlapping multiplier bits at a time and produces one of the digits $\{-2, -1, 0, +1, +2\}$ per step, halving the number of cycles (roughly $n/2$ instead of n) for the same operand width — this is the standard optimization used in hardware multipliers to reduce execution cycles while keeping the same correctness guarantees demonstrated above.

Step 4 – Cross-Verification

As an independent check, direct multiplication gives $13 \times 9 = 117$, and since exactly one operand (-13) is negative, the true product must be negative: -117 . This matches the value obtained from the corrected Booth trace ($A:Q = 1110001011 = -117$), confirming the fix is correct.

Debugging Checklist for Booth Implementations:

(a) Confirm the shift of $[A \ Q \ Q-1]$ is arithmetic, not logical, in every cycle. (b) Confirm $Q-1$ is initialised to 0 before the first cycle. (c) Confirm the operand width is large enough to represent both the multiplicand and multiplier in 2's complement without truncation (5 bits were required here, not 4). (d) Confirm the loop runs exactly n cycles for n -bit operands, since running one cycle short or long shifts every subsequent bit interpretation.

Question 2: Debugging Restoring Division ($29 \div 5$)

Problem Understanding:

The restoring division algorithm is giving an incorrect quotient for Dividend = 29 (11101) and Divisor = 5 (00101), using 5-bit registers A=00000, Q=11101, M=00101. We re-execute the algorithm correctly to find where the restore step was implemented incorrectly.

Step 1 – Correct Cycle-by-Cycle Execution

Cycle 1: shift [A,Q] left $\rightarrow$ A=00001, Q=11010. $A-M = 00001-00101 = \text{negative} \rightarrow$ RESTORE (A back to 00001), Q0=0 $\rightarrow$ Q=11010.

Cycle 2: shift $\rightarrow$ A=00011, Q=10100. $A-M = 00011-00101 = \text{negative} \rightarrow$ RESTORE (A back to 00011), Q0=0 $\rightarrow$ Q=10100.

Cycle 3: shift $\rightarrow$ A=00111, Q=01000. $A-M = 00111-00101 = 00010$ (non-negative) $\rightarrow$ keep A=00010, Q0=1 $\rightarrow$ Q=01001.

Cycle 4: shift $\rightarrow$ A=00100, Q=10010. $A-M = 00100-00101 = \text{negative} \rightarrow$ RESTORE (A back to 00100), Q0=0 $\rightarrow$ Q=10010.

Cycle 5: shift $\rightarrow$ A=01001, Q=00100. $A-M = 01001-00101 = 00100$ (non-negative) $\rightarrow$ keep A=00100, Q0=1 $\rightarrow$ Q=00101.

Final: Remainder = A = 00100 = 4, Quotient = Q = 00101 = 5. Check: $5 \times 5 + 4 = 29 \checkmark$.

Step 2 – Locating the Faulty Restore Operation

The buggy implementation was restoring A using the value of A AFTER it had already been shifted for the NEXT cycle, instead of restoring it immediately to the value it held just before the failed trial subtraction in the CURRENT cycle. Concretely, in Cycle 2 above, the correct restore must bring A back to 00011 (its value right after the Cycle-2 shift, before subtracting M); the faulty code instead added M to the already-shifted value from Cycle 3, one step too late. This one-cycle delay in the restore corrupts every quotient bit computed from Cycle 2 onward, because each subsequent trial subtraction starts from the wrong remainder.

Step 3 – Optimized Correction

Fix: perform the comparison and restore atomically within the same cycle — compute $A-M$, test its sign immediately, and if negative, restore A to its pre-subtraction value in that same cycle before shifting again (as done correctly in Step 1). A cycle-efficient alternative is to compute the trial subtraction into a temporary register and only commit it to A when non-negative, avoiding a separate 'undo' step altogether; this removes the possibility of a delayed/incorrect restore and matches the correct result: Quotient = 5, Remainder = 4.

Step 4 – Cross-Verification

Direct division confirms $29 \div 5 = 5$ remainder 4 (since $5 \times 5 = 25$ and $29 - 25 = 4$), exactly matching the corrected trace. This agreement between the bit-level algorithm and ordinary long division confirms the restore-timing fix removes the bug.

Debugging Checklist for Restoring Division:

(a) Restore must use the exact pre-subtraction value of A from the SAME cycle, never a value carried over from a later shift. (b) The quotient bit Q0 must be set in the same cycle as

the restore decision, not deferred. (c) Register width must be sufficient to hold the dividend and the partial remainders without overflow — 5 bits were required here since 29 exceeds 4-bit range. (d) The number of cycles must equal the number of quotient bits desired.

Question 3: Debugging Non-Restoring Division (after the 4th Iteration)

Problem Understanding:

A non-restoring division routine reports incorrect quotient bits after its fourth iteration. We use a representative 4-bit example — Dividend = 11 (1011), Divisor = 3 (0011), A=0000, Q=1011, M=0011 — to re-derive the correct 4-iteration trace and identify the logic error.

Step 1 – Correct Cycle-by-Cycle Execution

Rule: shift [A,Q] left by 1; if the PREVIOUS cycle's A was non-negative, subtract M; if it was negative, add M; then set Q0 = 1 if the new A is non-negative, else Q0 = 0.

Iteration 1: shift → A=0001; previous A (initial) ≥ 0 → subtract: A=0001–0011=1110 (negative) → Q0=0 → Q=0110.

Iteration 2: shift → A=1100; previous A < 0 → add: A=1100+0011=1111 (negative) → Q0=0 → Q=1100.

Iteration 3: shift → A=1111; previous A < 0 → add: A=1111+0011=0010 (non-negative) → Q0=1 → Q=1001.

Iteration 4: shift → A=0101; previous A ≥ 0 → subtract: A=0101–0011=0010 (non-negative) → Q0=1 → Q=0011.

Since the final A (0010) is non-negative, no end correction is needed. Final: Remainder = A = 0010 = 2, Quotient = Q = 0011 = 3. Check: $3 \times 3 + 2 = 11$ ✓.

Step 2 – Locating the Logic Error

The faulty code decided add-vs-subtract by testing the sign of A AFTER the left shift in the current iteration rather than the sign carried over from the END of the PREVIOUS iteration. Because the shift operation changes A's bit pattern (and can even change its sign bit), testing the post-shift A instead of the pre-shift/previous A picks the wrong operation on any iteration following a negative result — this is exactly what corrupts Iteration 4 in a buggy trace: Iteration 3 correctly ends with a non-negative A, but if Iteration 4 mistakenly re-checks a stale or freshly-shifted value instead of that carried-over non-negative flag, it performs an add instead of a subtract, giving Q0=0 and a wrong final quotient bit (and hence a wrong quotient/remainder pair).

Step 3 – Correction and Verification

Fix: latch the sign of A at the END of each iteration into a 1-bit flag, and use that stored flag — not a freshly re-evaluated value — to choose add or subtract at the START of the next iteration, exactly as applied in Step 1. Re-running the corrected algorithm on $11 \div 3$ gives the verified result Quotient = 3, Remainder = 2, matching $3 \times 3 + 2 = 11$.

Step 4 – Cross-Verification

Ordinary division confirms $11 \div 3 = 3$ remainder 2 ($3 \times 3 = 9$, $11 - 9 = 2$), matching the corrected trace exactly. Because the final A after Iteration 4 is non-negative, no end-of-algorithm correction step is required here — this only becomes necessary when the last iteration ends with a negative A (as seen in Question 2 of Assessment Tool 1's Q4 style non-restoring example).

Debugging Checklist for Non-Restoring Division:

(a) The add/subtract decision must always be based on the sign carried over from the previous iteration's result, latched in a flag, never re-derived from a post-shift value. (b) A single end-correction (add M back) is needed only if the very last iteration's A is negative — omitting this check, or applying it every iteration regardless of sign, is another common source of an incorrect final remainder. (c) Quotient bits must be recorded in the same iteration the sign is evaluated, so bit positions stay aligned with the corresponding shift.

Question 4: Arithmetic vs Logical Shift in a Booth Multiplier

Problem Understanding:

A Booth multiplier is shifting the $[A\ Q\ Q-1]$ register logically (filling the vacated most-significant bit with 0) instead of arithmetically (replicating the sign bit), which corrupts signed multiplication results whenever the partial product A is negative.

Step 1 – Why the Distinction Matters

In 2's complement, a negative number's magnitude is encoded across all its bits, including the sign bit; halving a negative value correctly (which is what a right shift during Booth's algorithm represents) requires the new MSB to remain 1 so the number stays negative and scales properly by powers of two. A logical shift instead always inserts 0 at the MSB, which silently turns a negative partial product into a (much smaller) positive one.

Step 2 – Worked Illustration (reusing -13×9 from Question 1)

At Cycle 2 of the -13×9 trace, $A = 11001$ (a negative value, -7) just before shifting.

Correct arithmetic shift: sign bit (1) is copied into the new MSB $\rightarrow A$ becomes 11100 (-4), which is exactly -7 halved and rounded toward $-\infty$, as required.

Incorrect logical shift: the new MSB is forced to 0 $\rightarrow A$ becomes 01100 ($+12$), a completely wrong value with the wrong sign and wrong magnitude.

This single wrong bit propagates through every remaining cycle's add/subtract decisions (since Q_0Q_{-1} pairs are unaffected, but the accumulator values used in the additions/subtractions are now wrong), so the buggy logical-shift version produces a final product nowhere near the correct -117 .

Step 3 – Correction

Fix: implement the shift of the combined $[A\ Q\ Q-1]$ register as an arithmetic right shift, i.e., the new MSB of A must always equal A's own MSB from before the shift (sign extension), for every cycle, not just the ones where it happens to be 0. This guarantees that negative partial products remain correctly signed throughout execution, which is essential because Booth's algorithm relies on 2's complement arithmetic at every intermediate step, not only in the final result.

Step 4 – Broader Impact and Prevention

This class of bug is easy to miss in testing because it is silent whenever A happens to be non-negative (logical and arithmetic shifts behave identically for a positive or zero A). It only surfaces once a multiplicand/multiplier combination drives A negative mid-computation, which is common whenever the multiplicand is negative (as in -13×9) or when a subtraction step temporarily overshoots. A reliable prevention strategy is to explicitly unit-test Booth's algorithm with at least one negative multiplicand, one negative multiplier, and one case with both operands negative, rather than relying only on all-positive test vectors, since the latter can pass even with a logical-shift bug present.

Question 5: Debugging IEEE 754 Exponent Alignment (Floating-Point Addition)

Problem Understanding:

A single-precision floating-point adder gives an incorrect sum because its exponent-alignment step is implemented incorrectly. We illustrate with $A = 12.5$ and $B = 0.75$.

Step 1 – Normalize Both Operands

$12.5 = 1100.1 = 1.1001 \times 2^3$ (sign 0, true exponent 3).

$0.75 = 0.11 = 1.1 \times 2^{-1}$ (sign 0, true exponent -1).

Step 2 – Correct Alignment Procedure

The exponents differ by $3 - (-1) = 4$. The rule is: the operand with the SMALLER exponent (0.75, exponent -1) has its mantissa shifted RIGHT by the difference (4 bits) so both operands share the LARGER exponent (3): 1.1×2^{-1} becomes 0.00011×2^3 .

Adding at the common exponent 3: $1.10010 + 0.00011 = 1.10101 \times 2^3 = 1101.01$ (binary) = 13.25, which matches the exact expected result $12.5 + 0.75 = 13.25$.

Step 3 – Locating the Bug

The faulty program was shifting the mantissa of the LARGER-exponent operand (12.5) to the right instead of the smaller one, and/or computing the shift amount as (smaller exponent – larger exponent), giving a negative shift count that the code silently treated as zero. Either fault causes the two mantissas to be added while still representing different powers of two — e.g., adding 1.1001 (meant to be $\times 2^3$) directly to 1.1 (meant to be $\times 2^{-1}$) without any shift produces a mantissa sum that is wrongly interpreted at scale 2^3 , yielding a result far larger or smaller than the true 13.25.

Step 4 – Optimized Correction

Fix: always compute $\text{shift} = |E_{\text{larger}} - E_{\text{smaller}}|$, identify which operand has the smaller exponent by comparison (not by input order), and right-shift only that operand's mantissa by 'shift' bits, adopting the larger exponent as the result's exponent before adding. To optimize this in hardware/software, compute the exponent difference once, clamp it to the mantissa width (further shifting is unnecessary once the smaller operand's mantissa is fully shifted out, i.e., its contribution becomes 0), and use guard/round/sticky bits captured during the shift to round the final normalized sum correctly — this avoids both the misalignment bug and unnecessary extra shift cycles.

Step 5 – Post-Addition Normalization Check

After adding the aligned mantissas, the result 1.10101×2^3 already has a single leading 1 before the binary point, so it is already normalized and no further shifting of the result is required in this example. In general, if the mantissa addition produces a carry-out (result ≥ 2.0), the sum must be shifted right by one bit and the exponent incremented by one; if the two operands are close in magnitude and opposite in effective sign, the result can instead have leading zeros, requiring a left shift and a corresponding exponent decrement — omitting either of these post-addition normalization checks is a second, related bug commonly found alongside the alignment bug described above.

Debugging Checklist for Floating-Point Alignment:

(a) Always shift the mantissa belonging to the SMALLER exponent, determined by explicit comparison, never by assuming input order. (b) Compute the shift amount as a non-negative difference; a negative shift count is itself a sign that the comparison logic is wrong. (c) Re-normalize the sum after addition (shifting and adjusting the exponent as needed) before packing the result back into sign/exponent/mantissa fields. (d) Preserve guard, round and sticky bits during the shift so that rounding decisions remain accurate.

Summary of Debugging Approach Across All Questions

Across all five problems, the recurring engineering principle is the same: signed/2's-complement and floating-point hardware algorithms are correct only when (1) shifts that must preserve sign (arithmetic shifts) are not silently replaced by shifts that do not (logical shifts), (2) state carried between iterations — a restore value, a sign flag, an exponent comparison — is captured and reused precisely once, in the correct cycle, rather than recomputed from a stale or already-mutated register, and (3) register/field widths are chosen large enough to represent every intermediate value without truncation. Each corrected trace above was independently cross-checked against ordinary decimal arithmetic (-117 , 5 remainder 4, 3 remainder 2, and 13.25) to confirm the fix removes the bug rather than merely changing its symptoms.